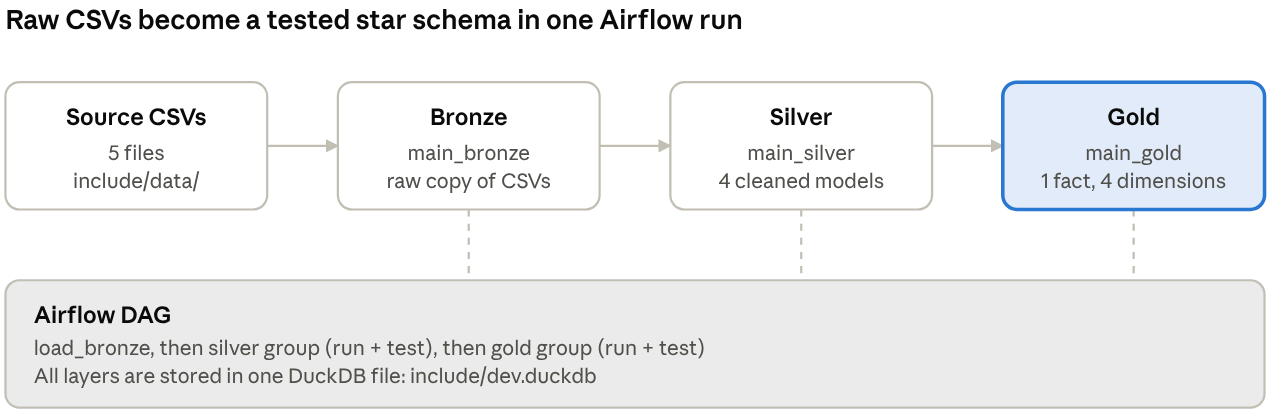
# Sales CRM Data Warehouse: Airflow + dbt + DuckDB

Oct 10, 2026 · @Mahmoud

## Overview

This project builds a sales CRM data warehouse from five CSV files. Airflow orchestrates the run, dbt transforms the data, and DuckDB stores the result in a single file, `dev.duckdb`.

The data follows a medallion architecture: raw CSVs land in bronze, are cleaned in silver, and are modelled as a star schema in gold. Each run is a Python load task followed by dbt run and test steps per layer.




pipeline overview · 4 layers run by one Airflow DAG

Data moves left to right through the layers, and one Airflow DAG runs and tests each layer in order.

| Component | Choice | Role |
|---|---|---|
| Orchestrator | Apache Airflow 2 on Astro Runtime 12.6.0 | Schedules and monitors the pipeline |
| Transformations | dbt-core 1.12.5 with dbt-duckdb 1.11.0 | Silver and gold models, tests, docs |

| Component | Choice | Role |
|---|---|---|
| Warehouse | DuckDB (file `dev.duckdb`) | Stores bronze, silver and gold schemas |
| dbt integration | Astronomer Cosmos | Turns each dbt model and test into an Airflow task |
| Runtime | Docker Desktop and Astro CLI | Runs Airflow locally in containers |
| Modelling | ER diagrams (draw.io style) | Conceptual, logical and physical models |

The project was built in two orchestration versions: first with `BashOperator` (four large tasks), then with Cosmos (one task per model and test). Both write to the same database, so they must not run at the same time.

## Project structure

The Astro project lives in `airflow_project/`. The dbt project sits inside `include/dbt/`, so the scheduler does not scan its files as DAGs.

```
airflow-dbt_project/
├── .venv/                        dbt on your Windows machine (local runs)
└── airflow_project/              Astro project root
    ├── dags/
    |   ├── bashoperator_dbt_dag.py   DAG with BashOperator tasks
    |   └── cosmos_dbt_dag.py         DAG with Cosmos task groups
    ├── include/
    |   ├── bronze.py                 load_bronze() function
    |   ├── data/                     the five source CSVs
    |   ├── dev.duckdb                database used inside the container
    |   └── dbt/dbt_project/
    |       ├── dbt_project.yml
    |       ├── profiles.yml          targets: dev and local
    |       ├── models/silver/        cleaned models + schema.yml
    |       ├── models/gold/          star schema + schema.yml
    |       ├── models/overview.md    docs landing page
    |       └── tests/silver, tests/gold   singular tests
```

```
    ├── plugins/                    unused (custom operators)
    ├── Dockerfile                  builds dbt_venv
    ├── requirements.txt            astronomer-cosmos, duckdb
    └── .dockerignore
```

| Folder or file | What goes in it |
|---|---|
| `dags/` | DAG definitions only. The scheduler parses every `.py` file here. |
| `include/` | Helper code, data, the database and the dbt project. Not parsed as DAGs. |
| `plugins/` | Custom operators, hooks and sensors. Empty in this project. |
| `Dockerfile` | Runtime image plus the isolated `dbt_venv` with `dbt-duckdb`. |
| `requirements.txt` | Python packages for the Airflow environment. |
| `.dockerignore` | Keeps `.venv`, `*.duckdb`, dbt `target/` and `logs/` out of the image. |

## Two virtual environments

`.venv` and `dbt_venv` both hold the dbt program, not the project. The project (models and tests) is shared by both.

| | `.venv` | `dbt_venv` |
|---|---|---|
| Location | Your Windows machine, outside `airflow_project` | Inside the container at `/usr/local/airflow/dbt_venv` |
| Created by | `python -m venv .venv` | The `Dockerfile` |
| Used by | You, in the terminal (target `local`) | Airflow tasks (target `dev`) |

The `dbt_venv` isolates dbt from Airflow so their dependencies do not conflict.

# Source data and bronze layer

The bronze layer is an unmodified copy of five CSV files, loaded into the `main_bronze` schema by `include/bronze.py`.

| Bronze table | Source file | Content |
|---|---|---|
| `accounts` | `accounts.csv` | Company, sector, year established, revenue, employees, office location, parent company |
| `products` | `products.csv` | Product, series, sales price |
| `sales_pipeline` | `sales_pipeline.csv` | One row per opportunity: agent, product, account, deal stage, engage and close dates, close value |
| `sales_team` | `sales_teams.csv` | Sales agent, manager, regional office |
| `data_dictionary` | `data_dictionary.csv` | Field descriptions for the other files |

## How the load works

`load_bronze()` connects to `dev.duckdb`, creates the schema if needed, and runs `CREATE OR REPLACE TABLE main_bronze.<name> AS SELECT * FROM read_csv_auto(...)` for each file. DuckDB infers the column types from the data. The connection is closed in a `finally` block, so a failed load never leaves the file locked for dbt.

Paths come from `AIRFLOW_HOME` (`/usr/local/airflow` in the container): data is read from `include/data/` and written to `include/dev.duckdb`. The Airflow task imports the function from `include/`, added to `sys.path` in the DAG.

The dbt source is declared once as `sales_crm`, pointing at schema `main_bronze` with the five tables above.

## Data quality issues found

| Issue | Where | Fix |
|---|---|---|
| Sector misspelled as `technolgy` | `accounts.sector` | Corrected to `technology` in silver |

| Issue | Where | Fix |
|---|---|---|
| Product written as `GTXPro` instead of `GTX Pro` | `sales_pipeline.product` | Standardised in silver so it matches `products` |
| Parent company stored as a name, not an ID | `accounts.subsidiary_of` | Kept as a name; `is_parent` is derived in gold |
| No keys in the source dimensions | accounts, products, sales team | Surrogate keys generated in gold |
| Open deals have no close date | `sales_pipeline.close_date` | Left NULL; documented in schema |

## Silver layer

Silver holds four cleaned models, one per business table, built from the bronze source `sales_crm`. They fix naming and spelling problems without changing the grain of the data.

| Model | Source table | Transformations |
|---|---|---|
| `accounts` | `accounts` | Corrects `technolgy` to `technology`; keeps `subsidiary_of` as NULL when the account has no parent |
| `products` | `products` | Renames `product` to `product_name` and `series` to `product_series` |
| `sales_pipeline` | `sales_pipeline` | Renames `product` to `product_name`; maps `GTXPro` to `GTX Pro` |
| `sales_team` | `sales_team` | Selects agent, manager and regional office |

The `data_dictionary` table stays in bronze as reference material and has no silver model.

Every silver model is described column by column in `models/silver/schema.yml`, including the notes that `subsidiary_of` and `close_date` can be NULL on purpose. The same file holds the silver generic tests, listed in the testing section.

Two naming rules keep the layers consistent: model file names match the names used in `ref()` (`sales_team`, not `sales_teams`), and the join keys between silver tables are the cleaned text values (account, product name, agent name).

## Gold layer and data models

Gold is a star schema with one fact table, where each row is a sales opportunity, and four dimensions. It lives in the `main_gold` schema and is built as tables.

| Model | Grain | Primary key | Notes |
|---|---|---|---|
| `fct_sales_opportunity` | One row per sales opportunity | `opportunity_id` | Holds five foreign keys, `deal_stage` and `close_value` |
| `dim_accounts` | One row per company | `account_id` | `is_parent` is 1 when another account lists it in `subsidiary_of` |
| `dim_products` | One row per product | `product_id` | Name, series, sales price |
| `dim_sales_team` | One row per sales agent | `agent_id` | Agent, manager, regional office |
| `dim_dates` | One row per date | `date_key` | Built from the distinct engage and close dates |

### Design decisions

- **Surrogate keys** use `ROW_NUMBER() OVER (ORDER BY name)`. This gives small integer IDs instead of long hashes. The IDs are rebuilt on every run, so nothing outside the pipeline should store them.
- **`dim_dates` is a role-playing dimension.** The fact joins it twice: `engage_date_key` and `close_date_key`. The key is an integer in `YYYYMMDD` format.
- **`close_date_key` is NULL for open deals** (Prospecting and Engaging). This is intentional and documented.
- **The fact joins the dimensions with left joins on cleaned names**, so no opportunity is dropped. A row-count test checks that the fact has as many rows as silver `sales_pipeline`.

- **The model is not tied to DuckDB.** In the diagrams, integer columns are drawn as `INT` and decimals as `FLOAT`, so the design reads the same on other warehouses.

## Conceptual model

Five entities: Account, Product, Sales Agent, Date and Sales Opportunity. The Opportunity is the central entity, and every relationship reads from it.

| Relationship | Dimension side | Opportunity side |
|---|---|---|
| Opportunity is for Account | exactly one | zero or many |
| Opportunity is about Product | exactly one | zero or many |
| Opportunity is managed by Agent | exactly one | zero or many |
| Opportunity is engaged on Date | exactly one | zero or many |
| Opportunity is closed on Date | zero or one | zero or many |

## Logical model

The logical model adds attributes, primary keys and foreign keys to the same five relationships, drawn with crow's foot notation. The fact table carries the note "each row represents a sales opportunity" next to it.

## Physical model

Column types as created in DuckDB (checked against `information_schema.columns`). Every column is nullable at the database level, because dbt creates tables with `CREATE TABLE AS SELECT`.

| Table | Columns and types |
|---|---|
| `dim_accounts` | account_id BIGINT, account_name VARCHAR, sector VARCHAR, year_established BIGINT, revenue DOUBLE, employees BIGINT, office_location VARCHAR, subsidiary_of VARCHAR, is_parent INTEGER |
| `dim_products` | product_id BIGINT, product_name VARCHAR, product_series VARCHAR, sales_price BIGINT |

| Table | Columns and types |
|---|---|
| `dim_sales_team` | agent_id BIGINT, agent_name VARCHAR, manager_name VARCHAR, regional_office VARCHAR |
| `dim_dates` | date_key INTEGER, date DATE, year BIGINT, month BIGINT, day BIGINT, month_name VARCHAR, day_name VARCHAR |
| `fct_sales_opportunity` | opportunity_id VARCHAR, account_id BIGINT, product_id BIGINT, agent_id BIGINT, engage_date_key INTEGER, close_date_key INTEGER, deal_stage VARCHAR, close_value BIGINT |

Primary keys, foreign keys, uniqueness and not-null rules are enforced by dbt tests, not by DuckDB constraints. The physical diagram shows this in a note and lists the constraints beside each table.

## Testing strategy

The project runs 45 dbt data tests in two kinds: generic tests declared in each layer's `schema.yml`, and singular tests written as SQL files in `tests/`. A test passes when its query returns zero rows.

### Generic tests

| Test | What it checks | Where it is used |
|---|---|---|
| `unique` | No repeated values | Primary keys and natural keys: `account`, `product_name`, `opportunity_id`, `account_id`, `date_key` and others |
| `not_null` | No NULL values | Same columns as `unique`, plus `sales_price` in silver |
| `relationships` | Every foreign key exists in the parent table; NULLs are ignored | Silver pipeline to accounts, products and sales team; gold fact to every dimension |

| Test | What it checks | Where it is used |
|---|---|---|
| `accepted_values` | Values come from a fixed list | `deal_stage` (Prospecting, Engaging, Won, Lost) and `is_parent` (0, 1) |

Foreign keys have no `not_null` test on purpose, because open deals can lack a close date.

## Singular tests

Each file in `tests/` holds one business rule and starts with a comment that states its purpose.

| File | Layer | Fails when |
|---|---|---|
| `assert_close_after_engage.sql` | silver | A deal closes before it was engaged |
| `assert_fact_rowcount_matches_pipeline.sql` | gold | The fact row count differs from silver `sales_pipeline` |
| `assert_no_unmatched_products.sql` | gold | A pipeline product has no match in `dim_products` |
| `assert_won_deals_have_close_date.sql` | gold | A Won deal lacks a close date or value |
| `assert_open_deals_have_no_close_date.sql` | gold | A Prospecting or Engaging deal has a close date |
| `assert_no_negative_values.sql` | gold | A `close_value` is negative |
| `assert_parent_flag_consistent.sql` | gold | `is_parent` disagrees with the actual subsidiaries |

## Selecting tests per layer

Silver runs 14 tests and gold runs 31. A singular test is picked up by any selector that covers a model it references, so gold tests that reference silver models were also selected in the silver step and failed because gold did not exist yet. The silver step therefore excludes `path:tests/gold`.

```
# silver
dbt test --select path:models/silver path:tests/silver --exclude
path:tests/gold
# gold
dbt test --select path:models/gold path:tests/gold
```

With Cosmos the same `select` and `exclude` values are passed to `RenderConfig`.

## Orchestration with Airflow

Two DAGs run the same pipeline: `bronze_dbt_bashoperator` treats dbt as a command, and `bronze_dbt_cosmos` turns every dbt model and test into its own task. Both use `schedule=None` (manual trigger) and `max_active_tasks=1`, because DuckDB allows only one writer.

### DAG 1: BashOperator

Five tasks run in order: `load_bronze`, `dbt_run_silver`, `dbt_test_silver`, `dbt_run_gold`, `dbt_test_gold`. Each dbt task runs the `dbt` executable from `dbt_venv` with `--profiles-dir . --target dev`. Airflow sees each layer as one box, so a failure points to a layer, not a model.

```
cd /usr/local/airflow/include/dbt/dbt_project && \
/usr/local/airflow/dbt_venv/bin/dbt run --select path:models/silver --
profiles-dir . --target dev
```

### DAG 2: Cosmos

`load_bronze` runs first, then a `DbtTaskGroup` named `silver`, then one named `gold`. Inside each group every model is a task, followed by its tests (`TestBehavior.AFTER_EACH`). The Airflow graph mirrors the `ref()` lineage, and a single failed model can be retried alone.

Cosmos is configured through four objects, shared by both groups except `RenderConfig`:

| Config | Question it answers | Value in this project |
|---|---|---|
| `ProjectConfig` | Where is the dbt project? | `/usr/local/airflow/include/dbt/dbt_project` and its `manifest.json` |

| Config | Question it answers | Value in this project |
|---|---|---|
| `ProfileConfig` | How does dbt connect? | Profile `dbt_project`, target `dev`, from `profiles.yml` |
| `ExecutionConfig` | Where is dbt installed? | `/usr/local/airflow/dbt_venv/bin/dbt` |
| `RenderConfig` | What becomes tasks? | `select` and `exclude` per layer, manifest load mode, tests after each model |

## Manifest instead of dbt ls

By default Cosmos runs `dbt ls` each time the scheduler parses the DAG file. That took longer than the 30 second limit and caused a `DagBag import timeout`. The fix is `LoadMode.DBT_MANIFEST`: Cosmos reads `target/manifest.json`, which `dbt parse` generates, so parsing takes seconds.

The manifest is a snapshot. After adding or changing a model, test or `ref()`, run `dbt parse` again, otherwise the DAG shows the old structure. The SQL of an existing model is always read fresh when the task runs.

## BashOperator compared with Cosmos

| | BashOperator | Cosmos |
|---|---|---|
| Tasks | 5 large tasks | One task per model and per test |
| Lineage in the UI | Hidden | Visible |
| Failure view | Read the layer's log | Failed model named in the graph |
| Retry | A whole layer | One model |
| Setup | Simple, no extra package | `astronomer-cosmos` and four configs |
| DAG parsing | Fast | Needs the manifest to stay fast |
| DuckDB concurrency | One task at a time by design | Needs `max_active_tasks=1` |

Cosmos also offers `DbtDag` (the whole DAG is dbt) and standalone operators. `DbtTaskGroup` fits here because bronze loading comes before dbt.

## Setup and how to run

You need Docker Desktop (running, with 6 to 8 GB of memory), the Astro CLI and Python on Windows. Start Docker before any `astro` command; a stopped Docker causes `exit status 1`.

### Configuration files

`Dockerfile`:

```
FROM quay.io/astronomer/astro-runtime:12.6.0

RUN python -m venv dbt_venv && source dbt_venv/bin/activate && \
    pip install --no-cache-dir --default-timeout=300 --retries 10 dbt-duckdb
&& deactivate
```

`requirements.txt`:

```
astronomer-cosmos
duckdb
```

`profiles.yml` (inside the dbt project):

```
dbt_project:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: /usr/local/airflow/include/dev.duckdb
      threads: 1
    local:
      type: duckdb
      path: ../../local.duckdb
      threads: 1
```

Target `dev` is used by Airflow inside the container. Target `local` is for running dbt on your machine; its path resolves to `include/local.duckdb`. Both use `threads: 1` because of DuckDB's single writer.

## One-time local setup

```
cd airflow-dbt_project
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install dbt-duckdb
```

## Start Airflow

Run from the Astro project root ( `airflow_project` ). The first start builds the image and can take several minutes.

```
astro dev start --wait 10m
astro dev run dags list-import-errors
```

The UI is at `localhost:8080` (default login `admin` / `admin` ). Trigger `bronze_dbt_cosmos` , or `bronze_dbt_bashoperator` . Do not run both at once.

## Common commands

| Task | Command | Run from |
|---|---|---|
| Generate the manifest for Cosmos | `dbt parse --profiles-dir . --target local` | `include/dbt/dbt_project` with `.venv` active |
| Run all tests locally | `dbt test --profiles-dir . --target local` | same |
| Preview a selector | `dbt ls --select "path:models/gold path:tests/gold" --profiles-dir . --target local` | same |
| Build and serve dbt docs | `dbt docs generate` then `dbt docs serve --port 8081` (with the same `--profiles-dir . --target local` ) | same |
| Shell inside the container | `astro dev bash` | `airflow_project` |

| Task | Command | Run from |
| --- | --- | --- |
| Reset containers and metadata | `astro dev kill` | `airflow_project` |

`astro dev kill` removes the containers and Airflow's metadata volume, not your files or `dev.duckdb`. Close DBeaver or any Python session that has `dev.duckdb` open before a run, or DuckDB reports a file lock.

The dbt docs show the lineage from bronze sources through silver to gold, with each test attached to its column. `models/overview.md` provides the landing page.

## Possible improvements

- Build `dim_dates` as a full calendar with `generate_series`, so there are no missing days.
- Add an Unknown member (key `-1`) to each dimension, and use it for open deals' close date.
- Replace the parent company name with a `parent_account_id` that points back to `dim_accounts`.
- Rename `dim_sales_team` to `dim_sales_agents`, since its grain is the agent.
- Use incremental models once data arrives continuously.
- Run `dbt test` and `dbt parse` in GitHub Actions on every push.